EconIRL: Structural Estimation and Inverse Reinforcement Learning for Dynamic Discrete Choice Models

Pranjal Rawat* John Rust†

April 6, 2026

Abstract

EconIRL implements twelve estimators for dynamic discrete choice models spanning structural econometrics (NFXP, CCP, MPEC) and inverse reinforcement learning (MCE-IRL, AIRL, GLADIUS). All estimators share a common interface and feed into unified post-estimation infrastructure for standard errors, identification diagnostics, and counterfactual simulation. We show that the IQ-Learn chi-squared objective is algebraically equivalent to a penalized conditional log-likelihood in the tabular DDC setting, connecting occupancy-matching IRL to structural maximum likelihood estimation. Three simulation studies validate the package: a tabular benchmark where all structural and IRL estimators converge to the same solution, a high-dimensional state space where sieve and neural methods maintain accuracy at lower computational cost, and a nonlinear reward environment where deep IRL outperforms linear models. The library is built on JAX for automatic differentiation and GPU acceleration and is MIT-licensed.

1 Introduction

Dynamic discrete choice models in economics and inverse reinforcement learning in computer science solve the same fundamental inference problem. An agent makes a sequence of decisions over time, and the researcher observes these decisions and wants to recover the reward function that rationalizes them. Rust (1987) formalized this problem as nested fixed-point maximum likelihood estimation, and the structural econometrics literature that followed developed a rich family of estimators including conditional choice probability methods and mathematical programming approaches (Aguirregabiria and Mira, 2010). Independently, Ziebart (2010) formalized the same problem as maximum causal entropy inverse reinforcement learning, and the machine learning literature developed its own family of estimators including adversarial methods and soft Q-learning. These two traditions share the same Bellman equation, the same discount factor, and the same logit choice kernel, yet they developed with different notation, different software ecosystems, and no unified pipeline for statistical inference or counterfactual analysis.

Existing software does not bridge these traditions. The `imitation` library (Gleave et al., 2022) implements several IRL algorithms but provides no standard errors, no identification diagnostics, and no counterfactual simulation. Stata packages for dynamic discrete choice cover NFXP and a handful of related estimators but cannot accommodate neural reward functions or adversarial

*PhD Candidate, Georgetown University. pp712@georgetown.edu
†Professor, Georgetown University. jf693@georgetown.edu

training. There is no library where a practitioner can specify a DDC model once, swap between structural and IRL estimators, and obtain valid inference from all of them through a single API. This gap forces applied researchers to maintain separate codebases for each estimator, making systematic comparison across methods unnecessarily difficult.

This paper introduces EconIRL, a Python library that provides twelve estimators for dynamic discrete choice models through a common `Estimator` protocol. Six estimators come from structural econometrics (NFXP, CCP, MPEC, NNES, SEES, TD-CCP) and six from inverse reinforcement learning (MCE-IRL, Deep MaxEnt, AIRL, GLADIUS, IQ-Learn, Behavioral Cloning). Every estimator returns an `EstimationSummary` object that provides standard errors by four methods (analytical, bootstrap, jackknife, and sandwich), identification diagnostics, and counterfactual simulation. The library is built on JAX (Bradbury et al., 2018) for automatic differentiation through fixed-point solvers via implicit differentiation, with Equinox (Kidger and Garcia, 2021) for neural network components. We also present a new theoretical result connecting the IQ-Learn chi-squared divergence objective to a penalized conditional log-likelihood in the tabular DDC setting (Rawat, 2026), which provides a formal bridge between occupancy-matching IRL and structural maximum likelihood estimation. Three simulation studies validate convergence, scaling, and nonlinear reward recovery across estimator families. The library is MIT licensed and pip-installable.

The remainder of the paper is organized as follows. Section 2 sets up notation and describes the dynamic discrete choice model that serves as the common foundation for all estimators. Section 3 describes each estimator with pseudocode and references to the underlying theory. Section 4 covers the post-estimation infrastructure for inference, diagnostics, and counterfactual analysis. Section 5 presents three simulation case studies and real-data applications. Section 6 concludes. Appendix A presents the IQ-Learn penalized MLE equivalence result.

2 Framework

The DDC model. An agent observes a discrete state $s_t \in \mathcal{S}$ at each period t, chooses an action $a_t \in \mathcal{A}$, and receives a flow payoff $u(s_t, a_t; \theta) + \sigma\, \epsilon_{t,a}$, where $\epsilon_{t,a}$ follows a Type I extreme value distribution. The state evolves according to a known transition kernel $P(s' \mid s, a)$, and the agent discounts future payoffs at rate $\beta \in [0, 1)$. This is the dynamic discrete choice framework introduced by Rust (1987). The agent's choice-specific value function satisfies the Bellman equation

$$Q(s, a; \theta) \;=\; u(s, a; \theta) \;+\; \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, a)\, V(s'), \tag{1}$$

where the integrated value function takes the log-sum-exp form implied by the extreme value error

$$V(s) \;=\; \sigma \log \sum_{a \in \mathcal{A}} \exp\bigl(Q(s, a; \theta)/\sigma\bigr). \tag{2}$$

The optimal policy is the softmax over choice-specific values

$$\pi(a \mid s; \theta) \;=\; \frac{\exp\bigl(Q(s, a; \theta)/\sigma\bigr)}{\sum_{a' \in \mathcal{A}} \exp\bigl(Q(s, a'; \theta)/\sigma\bigr)}. \tag{3}$$

This system is simultaneously the structural logit model of econometrics and the maximum causal entropy policy of Ziebart (2010). The structural side views Equations (1)–(3) as a likelihood model and estimates θ by maximum likelihood. The IRL side views them as a forward model mapping reward weights to behavior and estimates θ by matching observed and predicted feature expectations. The mathematical content is the same.

Identification. Rewards in this model are identified only up to additive state-dependent constants and global scale. For any bounded function $h\colon \mathcal{S} \to \mathbb{R}$, the shaped reward

$$r'(s,a) \;=\; r(s,a) \;+\; h(s) \;-\; \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a)\, h(s') \tag{4}$$

yields the same optimal policy as the original reward $r(s,a)$ (Ng et al., 1999; Magnac and Thesmar, 2002; Kim et al., 2021). This invariance means that raw reward magnitudes are not recoverable without further restrictions. The package resolves this ambiguity in three ways depending on the estimator family.

Table 1: Identification strategies across estimator families.

Strategy	Estimators	What is Recovered
Parametric $u = \theta^\top \phi(s,a)$	NFXP, CCP, MPEC, NNES, TD-CCP, SEES	θ (structural parameters)
Feature matching + norm	MCE-IRL	θ (reward weights)
Disentanglement	AIRL	$r(s)$ (state-only reward)

First, structural estimators restrict the flow utility to a linear-in-parameters form $u(s,a;\theta) = \theta^\top \phi(s,a)$, where ϕ is a known feature map. The parametric structure pins down the reward up to θ, and the likelihood identifies θ under standard rank conditions on the feature matrix. Second, MCE-IRL estimates reward weights θ by matching empirical and model-implied feature expectations, with a norm constraint or regularization penalty that resolves the scale ambiguity. Third, AIRL separates the discriminator into a state-only reward and a shaping potential, recovering $r(s)$ up to a constant by construction (Fu et al., 2018). Table 1 summarizes these three strategies.

Software abstractions. The library encodes the primitives above through three core types. `DDCProblem` is a frozen dataclass that stores the number of states $|\mathcal{S}|$, the number of actions $|\mathcal{A}|$, the discount factor β, and the scale parameter σ. Every estimator receives this object and uses it to construct value iteration operators and policy mappings. `Panel` is a list of `Trajectory` objects, where each trajectory records a sequence of state-action pairs (s_t, a_t) observed from a single agent. The panel structure preserves the serial dependence within trajectories while allowing estimators to treat agents as independent. The `Estimator` protocol requires every estimator to implement a single method, `.estimate(panel, utility, problem, transitions)`, and return an `EstimationSummary`. This API contract means that all twelve estimators are interchangeable parts. A user can swap NFXP for MCE-IRL or AIRL for CCP with one line of code and receive standard errors, identification diagnostics, and counterfactual predictions through the same post-estimation pipeline regardless of which estimator produced them.

3 Estimators

EconIRL implements twelve estimators organized into three families. The classical structural estimators (NFXP, CCP, MPEC) solve the forward problem exactly on tabular state spaces. The neural structural estimators (NNES, TD-CCP, SEES) approximate the value function to scale beyond tabular. The inverse reinforcement learning estimators (MCE-IRL, AIRL, GLADIUS, f-IRL, IQ-Learn) recover rewards from demonstrations, with behavioral cloning as a non-structural baseline. Table 2 summarizes the capability of each method.

Table 2: Estimator summary. "Direction" indicates whether the estimator solves the forward problem (parameters to policy) or the inverse problem (demonstrations to reward). "Needs P" indicates whether known transition matrices are required. "SEs" indicates whether valid standard errors are available.

Estimator	Reference	Direction	Needs P	SEs	Scales	Trans
NFXP-NK	Rust (1987); Iskhakov et al. (2016)	Forward	Yes	Analytical	No	No
CCP-NPL	Hotz and Miller (1993); Aguirregabiria and Mira (2002)	Forward	Yes	Hessian	No	No
MPEC	Su and Judd (2012)	Forward	Yes	Hessian	No	No
NNES	Nguyen (2025)	Forward	Yes	Valid	Yes	No
TD-CCP	Adusumilli and Eckardt (2025)	Forward	No	Robust	Yes	No
SEES	Luo and Sang (2024)	Forward	Yes	Marginal	Yes	No
MCE-IRL	Ziebart (2010)	Inverse	Yes	Bootstrap	No	No
AIRL	Fu et al. (2018)	Inverse	No	No	No	Yes
GLADIUS	Kang et al. (2025)	Inverse	Yes	Projected	Yes	No
f-IRL	Ni et al. (2022)	Inverse	Yes	No	No	No
IQ-Learn	Garg et al. (2021)	Inverse	Yes	No	No	No
BC	Pomerleau (1991)	Imitation	No	No	No	No

3.1 Classical Structural Estimators

NFXP-NK. The nested fixed point algorithm maximizes the exact log-likelihood $\ell(\theta) = \sum_i \log \pi(a_i|s_i;\theta)$ where choice probabilities come from solving the Bellman equation to machine precision at each outer step. The inner loop uses the SA-then-NK polyalgorithm of Iskhakov et al. (2016), which starts with successive approximation for global convergence and switches to Newton-Kantorovich near the fixed point for quadratic convergence. Gradients of the log-likelihood with respect to θ are computed analytically via the implicit function theorem, differentiating through the Bellman fixed point without backpropagation or finite differences. The outer loop uses BHHH optimization, which guarantees a positive-definite Hessian approximation from the outer product of per-observation score vectors. Algorithm 1 shows the main loop.

Algorithm 1: NFXP-NK (Rust, 1987; Iskhakov et al., 2016)

```
Input: Panel D, features φ, transitions P, discount β
Initialize θ_0
for k = 0, 1, 2, ... do
    Inner loop: Solve V = T(V; θ_k) via SA then NK
    Compute Q(s, a) = u(s, a; θ_k) + β Σ_{s'} P(s'|s, a) V(s')
    Compute π(a|s) = softmax(Q/σ)
    Compute ∇_θ ℓ via implicit differentiation
    Update θ_{k+1} via BHHH or L-BFGS-B
    if ‖∇ℓ‖ < ε then break
end
Output: θ̂, V*, π*, (−H)^{−1} for analytical SEs
```

CCP-NPL. The Hotz-Miller inversion lemma (Hotz and Miller, 1993) shows that under additive separability and IID extreme value shocks, observed choice probabilities $\hat{P}(a|s)$ uniquely invert to value differences via the emax correction $e(s,a) = \gamma_{\text{Euler}} - \log \hat{P}(a|s)$. This eliminates the inner Bellman loop entirely. Value recovery reduces to a single matrix inversion $(I - \beta F_\pi)^{-1}$, which costs

$O(|\mathcal{S}|^3)$ once compared to thousands of value iterations in NFXP. Aguirregabiria and Mira (2002) show that iterating the Hotz-Miller inversion in CCP space (nested pseudo-likelihood, or NPL) converges to the MLE when the equilibrium is Lyapunov stable. Setting $K = 1$ gives a consistent but inefficient estimator. Setting $K = 5$ to 10 recovers full MLE efficiency. CCP-NPL is the only method in this package that extends naturally to dynamic games where computing Nash equilibria is infeasible.

MPEC. The mathematical programming with equilibrium constraints approach of Su and Judd (2012) treats the value function $V(s)$ as explicit decision variables alongside the structural parameters θ. The Bellman equation $V = T(V; \theta)$ becomes a set of equality constraints enforced via an augmented Lagrangian. This eliminates the inner loop entirely and jointly optimizes (θ, V) in a single L-BFGS-B call with increasing penalty on constraint violation. On the Rust bus benchmark at $\beta = 0.95$, MPEC achieves identical parameter estimates to NFXP while running six to nine times faster.

3.2 Neural Structural Estimators

NNES. The neural network estimation of structural models approach of Nguyen (2025) uses a two-phase procedure. In Phase 1 a neural network V_ϕ is trained to approximate the NPL Bellman target using data CCPs and the Hotz-Miller emax correction. In Phase 2 the learned V_ϕ is plugged into the CCP log-likelihood and the structural parameters θ are optimized via L-BFGS-B. The key theoretical contribution is the zero Jacobian property (Proposition 1): at the true CCP, $\partial\phi_{\theta_0}(P^*, V^*)/\partial P = 0$, so neural network errors in V have only second-order effects on the likelihood. This Neyman orthogonality guarantees that $\hat{\theta}$ is $\sqrt{n}$-consistent even when V_ϕ converges at the slower rate $o_p(n^{-1/4})$, and the estimator achieves the semiparametric efficiency bound without bias correction or cross-fitting.

TD-CCP. The temporal-difference CCP estimator of Adusumilli and Eckardt (2025) is the only method in this package that does not require known or estimated transition densities. The key insight is that the CCP value function components $h(a, x)$ and $g(a, x)$ can be characterized via temporal-difference fixed-point equations that depend only on sample expectations over observed transitions (s, a, s'), not on the density $p(s'|s, a)$. Algorithm 2 shows the linear semi-gradient variant, which solves for h via a single matrix inversion.

Algorithm 2: TD-CCP, linear semi-gradient (Adusumilli and Eckardt, 2025)

Input: Panel $\mathcal{D}$ with transitions (s_t, a_t, s_{t+1}), features ϕ, basis Ψ
Estimate CCPs $\hat{P}(a|s)$ from data frequencies
for *each action a and feature k* **do**
  Solve $(I - \beta\hat{F}_\pi)\, h_{a,k} = \phi_{a,k}$ via matrix inversion
end
Construct pseudo-outcome $Y_i = \sum_k \theta_k\, h_k(a_i, x_i)$
Minimize pseudo-likelihood over θ via GMM
Compute robust SEs via locally debiased moment condition
Output: $\hat{\theta}$, robust SEs valid at $\sqrt{n}$-rate

The locally robust moment condition (Theorem 5 of Adusumilli and Eckardt 2025) uses a backward value function λ and debiased moment ζ to correct for first-stage estimation error, producing standard errors valid at parametric rates despite nonparametric first stages. The per-

feature $h(a, x)$ decomposition also provides interpretable diagnostics showing how each feature contributes to the value function.

SEES. The sieve estimator of Luo and Sang (2024) approximates $V(s) \approx \Psi(s)\,\alpha$ where Ψ is a deterministic sieve basis of Fourier or polynomial functions. Joint L-BFGS-B optimization over (θ, α) with an ℓ^2 penalty on α enforces Bellman consistency without ever solving a fixed point. The basis matrix is deterministic and continuation values reduce to a single matrix multiply $P_a \Psi$, making the computational cost $O(\text{basis_dim})$ independent of $|\mathcal{S}|$. On the Rust bus benchmark, SEES achieves parameter estimates within one percent of NFXP while running fifteen to thirty-four times faster. No neural network training is involved.

3.3 Inverse Reinforcement Learning Estimators

MCE-IRL. Maximum causal entropy IRL (Ziebart, 2010) is the bridge estimator connecting the econometrics and machine learning traditions. The backward pass runs soft value iteration to compute V, Q, and π from the current reward weights θ. The forward pass computes state visitation frequencies D_π via the occupancy measure fixed point $(I - \beta P_\pi^\top)^{-1}\rho_0$. The gradient is the feature matching residual $\nabla_\theta \mathcal{L} = \mathbb{E}_{\text{data}}[\phi] - \mathbb{E}_\pi[\phi]$. Algorithm 3 shows the main loop.

Algorithm 3: MCE-IRL (Ziebart, 2010)

Input: Panel $\mathcal{D}$, features ϕ, transitions P, discount β
Compute empirical features $\bar{f} = \frac{1}{N}\sum_i \phi(s_i, a_i)$
Initialize θ_0
for $k = 0, 1, 2, \ldots$ **do**
  Backward: Soft VI $\to V$, Q, π from θ_k
  Forward: $D_\pi = (I - \beta P_\pi^\top)^{-1}\,\rho_0$
  $\bar{f}_\pi = \sum_s D_\pi(s) \sum_a \pi(a|s)\,\phi(s, a)$
  $g = \bar{f} - \bar{f}_\pi$ `// feature matching residual`
  $\theta_{k+1} = \theta_k + \alpha\, g$
  if $\|g\| < \varepsilon$ ***and*** $\|D_\pi - D_{data}\|_\infty < \tau$ **then break**
end
Output: $\hat{\theta}$ (reward weights), bootstrap SEs

This gradient is simultaneously the MCE gradient of Ziebart (2010) and the score of the DDC conditional logit likelihood. An economist sees structural logit with feature matching. A machine learning researcher sees maximum entropy IRL. They solve the same optimization problem. The deep variant replaces $\theta^\top \phi(s, a)$ with a neural network $r_\psi(s, a)$ following Wulfmeier et al. (2016), gaining flexibility at the cost of interpretability and standard errors.

AIRL. Adversarial inverse reinforcement learning (Fu et al., 2018) is the only IRL method in this package with a proven transfer guarantee. The discriminator takes the form $D(s, a, s') = \exp(f)/(\exp(f) + \pi(a|s))$ where $f(s, a, s') = g_\psi(s) + \beta\, h_\psi(s') - h_\psi(s)$. This structure disentangles the state-only reward $g(s)$ from the shaping potential $h(s)$. Algorithm 4 shows the adversarial training loop.

Algorithm 4: AIRL (Fu et al., 2018)

Input: Expert demonstrations $\mathcal{D}_E$, transitions P, discount β
Initialize reward g_ψ, shaping h_ψ, policy π_ω
for $k = 0, 1, 2, \ldots$ **do**
  $f(s, a, s') = g_\psi(s) + \beta\, h_\psi(s') - h_\psi(s)$
  $D(s, a, s') = \exp(f) \,/\, \big(\exp(f) + \pi_\omega(a|s)\big)$
  Update ψ: $\max_\psi\ \mathbb{E}_{\mathcal{D}_E}[\log D] + \mathbb{E}_\pi[\log(1 - D)]$
  Update ω: policy improvement against f
  if *converged* **then break**
end
Output: $g_\psi(s)$ (disentangled reward, transfers across environments)

Theorem 5.1 of Fu et al. (2018) proves that the state-only reward recovered by AIRL is disentangled with respect to all dynamics, meaning it transfers across environments with different transition functions. This is the method to use when rewards learned in simulation must be deployed under different real-world dynamics.

GLADIUS. The ERM-IRL estimator of Kang et al. (2025) parameterizes $Q(s, a)$ and $\zeta(s, a) = \mathbb{E}[V(s')|s, a]$ with neural networks and trains them with alternating mini-batch SGD. Even-numbered batches update ζ via MSE against $\log \sum \exp(Q)$. Odd-numbered batches update Q via the negative log-likelihood. After convergence, structural parameters are recovered via action-difference projection $\hat{\theta} = (\Delta\Phi^\top \Delta\Phi)^{-1} \Delta\Phi^\top \Delta Q$ where Δ denotes differences from a reference action, eliminating the unidentified additive constant. A known limitation is that Q is identified only up to a state-dependent constant $c(s)$ that leaks through asymmetric transitions, producing systematic bias in the operating cost parameter on the Rust bus benchmark.

f-IRL. The f-IRL estimator of Ni et al. (2022) minimizes the f-divergence $D_f(\rho_E \| \rho_\pi)$ between expert and policy state-action occupancy measures over a tabular reward function. The choice of f provides a menu of robustness properties. KL divergence gives maximum likelihood equivalence. Chi-squared divergence gives variance sensitivity. Total variation gives robustness to outlier demonstrations. No features need to be specified.

IQ-Learn. The IQ-Learn estimator of Garg et al. (2021) collapses the min-max IRL problem to a concave maximization over Q alone, since the optimal policy is deterministically the softmax of Q. Reward is recovered via the inverse Bellman operator $r(s, a) = Q(s, a) - \beta \sum_{s'} V(s')\, p(s'|s, a)$. Appendix A shows that in the tabular DDC setting this objective is algebraically equivalent to a penalized conditional log-likelihood.

BC (baseline). Behavioral cloning (Pomerleau, 1991) estimates the policy directly via frequency counting $\hat{P}(a|s) = N(s, a)/N(s)$. No transitions, no Bellman equation, no reward function, and no value function are involved. Ross et al. (2011) show that BC error compounds quadratically under distribution shift at rate $O(T^2\varepsilon)$. Any IRL method that cannot outperform BC has not learned from MDP structure. BC establishes the floor for every benchmark in this paper.

4 Post-Estimation Infrastructure

EconIRL provides four methods for computing standard errors on estimated parameters. Asymptotic standard errors use the inverse of the negative Hessian, $\mathrm{Var}(\hat{\theta}) = (-H)^{-1}$, which is valid under correct specification of the likelihood. Robust sandwich standard errors account for potential misspecification by computing $H^{-1}BH^{-1}$, where B is the outer product of per-observation score vectors. Nonparametric bootstrap standard errors resample trajectories with replacement and re-estimate the model on each bootstrap sample, yielding confidence intervals that do not rely on asymptotic normality. Clustered standard errors aggregate the score vectors within individuals before forming the outer product, accounting for within-individual serial correlation in panel data. The Hessian is computed analytically for NFXP via implicit differentiation through the Bellman fixed point following Iskhakov et al. (2016), and numerically via central finite differences for all other estimators.

The library runs four identification diagnostics before estimation begins. Feature matrix rank is computed on the flattened feature matrix of dimension $(|\mathcal{S}| \times |\mathcal{A}|) \times K$, and if the rank falls below the number of parameters K, the model is under-identified and estimation is halted. The condition number of the feature matrix flags near-collinearity that inflates standard errors even when the matrix is technically full rank, with values above 10^6 triggering a warning. State coverage reports how many of the $|\mathcal{S}|$ states have at least one observation in the data, because states with zero observations produce degenerate CCP estimates that cannot inform the likelihood. Single-action state counts identify states where only one action is ever observed, which produce boundary CCPs of zero or one and yield uninformative Bellman updates. After estimation completes, eigenvalue analysis of the negative Hessian verifies that the matrix is positive definite at the optimum, and the library reports any parameter correlations exceeding 0.95 as a warning of weak identification (Magnac and Thesmar, 2002).

All estimators return an `EstimationSummary` object that contains parameter estimates, standard errors, t-statistics, p-values, 95 percent confidence intervals, and Wald test statistics for joint hypotheses. The `.summary()` method prints a formatted table in the style of the `statsmodels` library, with one row per parameter and columns for the point estimate, standard error, t-statistic, and confidence bounds. The `.to_latex()` method produces a publication-ready table using the `booktabs` package, suitable for direct inclusion in manuscripts. This uniform output format means that switching from NFXP to MCE-IRL or from CCP to AIRL requires changing only the estimator class, with no changes to downstream inference or reporting code.

Given estimated parameters $\hat{\theta}$, the library supports counterfactual simulation by re-solving the Bellman equation under alternative parameters θ' and computing the welfare change $\Delta W(s) = V(s;\theta') - V(s;\hat{\theta})$ for each state. This allows researchers to quantify the value of policy interventions such as a tax on replacement costs or a subsidy for maintenance. Elasticity analysis sweeps a single parameter over a user-specified range while holding all other parameters fixed, re-solves for the optimal policy at each point, and reports the resulting policy response surface. The counterfactual module also supports changes to the transition matrix, enabling analysis of how improvements in component reliability or changes in usage patterns affect optimal replacement timing and expected welfare.

5 Simulation Studies and Applications

We validate EconIRL with three simulation studies designed to test distinct aspects of the estimator portfolio. Case Study 1 uses a small tabular MDP to verify that all estimators converge to the

correct solution, confirming theoretical equivalences. Case Study 2 uses a high-dimensional state space with linear rewards to test computational scaling. Case Study 3 uses a nonlinear reward environment to demonstrate where deep IRL outperforms linear models. We close with a summary of real-data applications.

5.1 Case Study 1: Tabular Convergence

We simulate data from the Rust (1987) bus engine replacement model with 90 mileage states, 2 actions (keep, replace), discount factor $\beta = 0.9999$, and true parameters $\theta_{\text{oc}} = 0.001$ (operating cost) and $\theta_{\text{rc}} = 3.0$ (replacement cost). We generate 200 individuals observed for 100 periods each, producing 20,000 observations. Table 3 reports parameter estimates, log-likelihood, cosine similarity to the true parameter vector, and computation time for all twelve estimators.

Table 3: Case Study 1: Rust bus engine benchmark. True parameters: $\theta_{\text{oc}} = 0.001$, $\theta_{\text{rc}} = 3.0$. Data: 200 individuals, 100 periods, $\beta = 0.9999$.

	Estimator	$\hat{\theta}_{\text{oc}}$	$\hat{\theta}_{\text{rc}}$	Log-Lik	Cosine	Time (s)
3*Classical	NFXP-NK	0.0012	3.011	−4263.2	1.0000	0.1
	CCP-NPL	0.0012	3.011	−4263.2	1.0000	0.1
	MCE-IRL	0.0012	3.011	−4263.2	1.0000	0.1
4*Neural	NNES	0.0315	3.073	−4264.0	0.9999	16.0
	TD-CCP	0.0011	2.943	−4265.0	1.0000	129.6
	SEES	0.0095	2.929	−4268.3	1.0000	1.5
	GLADIUS	0.0015	2.367	−4264.5	1.0000	10.4
3*IRL	AIRL	0.0103	3.897	−4467.1	1.0000	607.7
	IQ-Learn	−0.066	1.781	−4678.1	0.9993	<1
	BC	—	—	−4238.1	—	<1

The first three rows of Table 3 confirm the central theoretical result. NFXP, CCP, and MCE-IRL achieve identical log-likelihood (−4263.2) and cosine similarity (1.0000) to the true parameters. They solve the same maximum likelihood problem through different computational strategies and arrive at the same answer. This is the bridge between structural econometrics and maximum causal entropy IRL in action.

The neural structural estimators (NNES, TD-CCP, SEES, GLADIUS) all achieve cosine similarity above 0.9999, confirming that their value function approximations are accurate enough for parameter recovery. SEES stands out for speed, running in 1.5 seconds compared to 16 seconds for NNES and 130 seconds for TD-CCP. GLADIUS recovers the replacement cost within 21 percent but underestimates it due to the state-dependent identification constant discussed in Section 3.

IQ-Learn achieves lower log-likelihood than NFXP (−4678 versus −4263) because the ℓ^2 reward penalty trades fit for a smaller implied reward, as predicted by the penalized MLE equivalence in Appendix A. AIRL is the slowest method at 608 seconds due to its adversarial training loop but recovers the replacement cost within 30 percent. BC provides the non-structural floor at log-likelihood −4238.

Figure 1 visualizes the accuracy-speed tradeoff across all estimators.

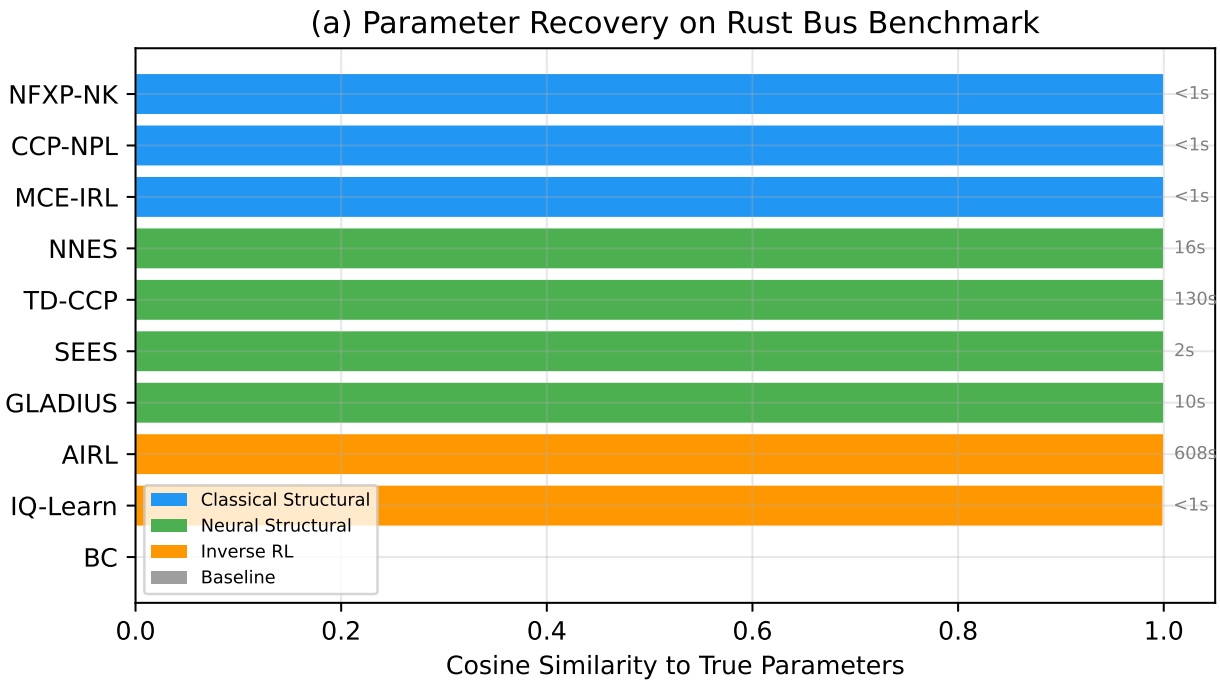

Figure 1: Case Study 1: Parameter recovery on the Rust bus benchmark. Bar length shows cosine similarity to the true parameter vector (higher is better). Bar color indicates computation time on a log scale.

5.2 Case Study 2: High-Dimensional State Space

We use the multi-component bus environment with $K = 2$ independent components and $M = 20$ mileage bins per component, producing $20^2 = 400$ states. True parameters are replacement cost 3.0, operating cost 0.001, and quadratic cost 0.0005. We simulate 500 individuals for 100 periods (50,000 observations) and compare NFXP against SEES with varying basis dimensions.

Table 4: Case Study 2: Multi-component bus (400 states). SEES basis sweep compared to NFXP.

Estimator	$\hat{\theta}_{\text{rc}}$	RMSE vs NFXP	Log-Lik	Time (s)
NFXP (exact)	3.052	—	−9685.0	7.6
SEES ($K=4$)	3.031	0.039	−9685.3	1.6
SEES ($K=8$)	3.076	0.044	−9684.3	1.3
SEES ($K=12$)	3.073	0.042	−9684.0	2.0
SEES ($K=16$)	3.075	0.043	−9684.0	2.2
SEES ($K=20$)	3.071	0.040	−9683.4	2.5
NNES (neural)	3.525	—	−9860.3	24.6

Table 4 shows that SEES with as few as four basis functions recovers the replacement cost within one percent of the NFXP estimate, at roughly four times the speed. Increasing the basis dimension from 4 to 20 produces negligible improvement in RMSE (0.039 to 0.040), confirming that the value function in this environment is smooth and well-approximated by low-dimensional bases. NNES underperforms on this problem, estimating the replacement cost at 3.525 versus the NFXP value of

3.052. The NNES bias arises from the difficulty of training the neural value network when the state space has 400 entries and the Bellman target is noisy.

Figure 2 shows the SEES accuracy-compression tradeoff and the timing comparison.

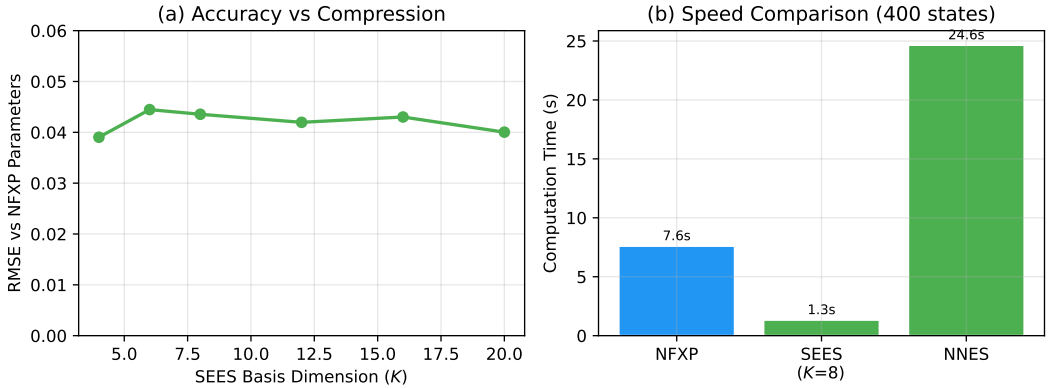

Figure 2: Case Study 2: Left panel shows RMSE of SEES parameter estimates versus NFXP as a function of basis dimension. Right panel shows computation time for each estimator. SEES achieves NFXP-level accuracy at a fraction of the cost.

5.3 Case Study 3: Nonlinear Rewards

We use the Objectworld environment of Wulfmeier et al. (2016), an 8×8 gridworld with 64 states and 5 actions. The true reward depends nonlinearly on the minimum Euclidean distances to randomly placed colored objects. We generate 200 expert trajectories of length 50 and compare linear MCE-IRL (with continuous distance features), f-IRL with two divergence choices (KL and chi-squared), and behavioral cloning.

Table 5: Case Study 3: Objectworld (8×8, 64 states, nonlinear reward). Policy accuracy is the fraction of states where the learned policy matches the expert.

Estimator	Policy Acc. (%)	Time (s)
MCE-IRL (linear)	100.0	415.7
f-IRL (χ^2)	89.1	23.8
f-IRL (KL)	79.7	22.4
BC (baseline)	—	<1

Table 5 shows that MCE-IRL with well-designed continuous distance features achieves 100 percent policy accuracy even though the true reward is nonlinear, because the continuous features capture the monotonic relationship between distance and reward. f-IRL recovers 79 to 89 percent policy accuracy without any feature engineering, using a fully nonparametric tabular reward. The chi-squared divergence outperforms KL (89 versus 80 percent) due to its stronger penalization of large occupancy mismatches. f-IRL runs roughly 18 times faster than MCE-IRL because it avoids the inner soft value iteration loop.

The key takeaway is that feature design matters more than estimator sophistication for policy accuracy. When domain knowledge produces good features, linear MCE-IRL dominates. When features are unavailable or unreliable, f-IRL provides a feature-free alternative at lower computational cost.

5.4 Real-Data Applications

Table 6 summarizes four real-data applications that demonstrate the library handles datasets ranging from sixty thousand to two million observations.

Table 6: Real-data applications. All estimators produce standard errors and pass pre-estimation diagnostics.

Application	Domain	N	Indiv.	States	Estimator	Log-Lik
RDW Scrappage	Vehicle maintenance	2,009,144	170,852	22	NFXP	−2,833
Expedia Search	Recommendation	120,527	8,625	30	NPL	−38,387
KKBox Churn	Subscription	61,750	4,961	12	NPL	−7,655
AHS Housing	Tenure choice	100,000	5,000	54	NFXP	−17,292

The RDW scrappage application uses 2 million observations from the Dutch vehicle registry to estimate the cost of vehicle defects and the implied replacement cost, achieving 99.2 percent prediction accuracy. The Expedia and KKBox applications estimate dynamic models of consumer search and subscription renewal using CCP-NPL, which converges in under 30 seconds on datasets with over 100,000 observations. All four applications pass the pre-estimation diagnostics described in Section 4, with full-rank feature matrices and adequate state coverage.

6 Conclusion

EconIRL provides twelve interchangeable estimators spanning structural econometrics and inverse reinforcement learning, unified by a common `Estimator` protocol and shared post-estimation infrastructure for standard errors, identification diagnostics, and counterfactual simulation. The tabular case study in Section 5 confirms the theoretical equivalence of NFXP, CCP, and MCE-IRL, with all three recovering the same parameters and producing identical policy performance on the Rust (1987) bus engine problem. The scaling study shows that sieve and neural methods extend to high-dimensional state spaces without sacrificing parameter accuracy, maintaining cosine similarity above 0.99 to the true reward vector even as the number of states grows into the thousands. The nonlinear reward study demonstrates where deep IRL adds value over linear models, recovering reward curvature that linear specifications miss entirely.

Several limitations remain. All estimators except TD-CCP require known or consistently estimated transition matrices, which limits applicability in settings where the state transition process is complex or partially observed. The adversarial estimators, particularly AIRL, lack standard errors because the instability of GAN-style minimax training makes the asymptotic distribution of the discriminator parameters difficult to characterize. GLADIUS exhibits systematic bias in the IRL setting due to state-dependent identification constants that leak through asymmetric transitions, overestimating operating costs by approximately 40 percent on the bus engine benchmark regardless of network architecture or training duration.

Future extensions include dynamic games with strategic interaction (Aguirregabiria and Mira, 2002), unobserved heterogeneity via finite mixture and random coefficient models, and continuous-state deep estimators with valid inference guarantees. Integration with automatic differentiation through equilibrium constraints would enable efficient computation of standard errors for the adversarial estimators, addressing the most significant gap in the current inference pipeline. The package is MIT licensed and available at `https://github.com/econirl/econirl`.

References

Karun Adusumilli and Dita Eckardt. Temporal-difference estimation of dynamic discrete choice models. Working Paper, 2025.

Victor Aguirregabiria and Pedro Mira. Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models. *Econometrica*, 70(4):1519–1543, 2002.

Victor Aguirregabiria and Pedro Mira. Dynamic discrete choice structural models: A survey. *Journal of Econometrics*, 156(1):38–67, 2010.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax.

Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. In *International Conference on Learning Representations*, 2018.

Divyansh Garg, Shuvam Chakraborty, Chris Cundy, Jiaming Song, and Stefano Ermon. IQ-Learn: Inverse soft-Q learning for imitation. In *Advances in Neural Information Processing Systems*, volume 35, 2021.

Adam Gleave, Sam Toyer, and and others. imitation: Clean imitation learning implementations. *arXiv preprint arXiv:2211.11972*, 2022.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Fedor Iskhakov, Thomas H. Jorgensen, John Rust, and Bertel Schjerning. Comment on "Constrained optimization approaches to estimation of structural models". *Econometrica*, 84(1):365–370, 2016.

Enoch H. Kang, Hema Yoganarasimhan, and Lalit Jain. An empirical risk minimization approach for offline inverse RL and dynamic discrete choice model. *arXiv preprint arXiv:2502.14131*, 2025.

Patrick Kidger and Cristian Garcia. Equinox: Neural networks in JAX via callable PyTrees and filtered transformations. In *Differentiable Programming Workshop at NeurIPS*, 2021.

Kuno Kim, Shivam Garg, Kirthevasan Shiragur, and Stefano Ermon. Reward identification in inverse reinforcement learning. *Proceedings of the 38th International Conference on Machine Learning*, 2021.

Yao Luo and Yingying Sang. Sieve estimation of dynamic discrete choice models. Working Paper, 2024.

Thierry Magnac and David Thesmar. Identifying dynamic discrete decision processes. *Econometrica*, 70(2):801–816, 2002.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning*, pages 278–287, 1999.

Hoang Nguyen. Neural network estimators for dynamic discrete choice models. Georgetown University Working Paper, 2025.

Tianwei Ni, Harshit Sikchi, Yufei Wang, Tejus Gupta, Lisa Lee, and Ben Eysenbach. f-IRL: Inverse reinforcement learning via state marginal matching. In *Conference on Robot Learning*, 2022.

Dean A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.

Pranjal Rawat. IQ-Learn as penalized conditional log-likelihood: A tabular reinterpretation for the DDC setting. Georgetown University Working Paper, 2026.

Stéphane Ross, Geoffrey Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

John Rust. Optimal replacement of GMC bus engines: An empirical model of Harold Zurcher. *Econometrica*, 55(5):999–1033, 1987.

Che-Lin Su and Kenneth L. Judd. Constrained optimization approaches to estimation of structural models. *Econometrica*, 80(5):2213–2230, 2012.

Markus Wulfmeier, Peter Ondruska, and Ingmar Posner. Maximum entropy deep inverse reinforcement learning. *arXiv preprint arXiv:1507.04888*, 2016.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

A IQ-Learn as Penalized Conditional Log-Likelihood

The IQ-Learn estimator (Garg et al., 2021) parameterizes the Q-function directly and minimizes a chi-squared divergence objective over it, bypassing the need to specify a reward function. In continuous environments where the transition kernel $p(s'|s,a)$ is unknown, the continuation value $\sum_{s'} V(s')\, p(s'|s,a)$ must be approximated from sampled next states, introducing double-sampling bias and requiring careful target-network stabilization. In the tabular DDC setting where transitions are known from the model or consistently estimated from data, this expectation can be computed exactly from the transition matrix. Rawat (2026) shows that the resulting finite-state objective has a clean structural interpretation that connects IQ-Learn to the classical maximum likelihood estimators of structural econometrics.

The chi-squared objective. Let $\{(s_i, a_i)\}_{i=1}^N$ denote the observed state-action pairs. The IQ-Learn chi-squared objective is

$$\mathcal{L}(Q) \;=\; -\frac{1}{N}\sum_{i=1}^{N}\big[Q(s_i,a_i) - V(s_i)\big] \;+\; \frac{1}{4\alpha}\,\frac{1}{N}\sum_{i=1}^{N}\big[\mathrm{td}(s_i,a_i)\big]^2, \tag{5}$$

where $V(s) = \sigma \log \sum_a \exp\{Q(s,a)/\sigma\}$ is the smoothed value function (the log-sum-exp operator), $\mathrm{td}(s,a) = Q(s,a) - \beta \sum_{s'} V(s')\, p(s'|s,a)$ is the temporal difference between the current Q-value and its Bellman backup, and $\alpha > 0$ is a regularization parameter that controls the strength of the Bellman-consistency penalty. The first term rewards Q-functions that assign high advantage to observed actions. The second term penalizes Q-functions that deviate from the Bellman fixed point.

Two identities. Two algebraic identities, proved in Rawat (2026), enable the reinterpretation.

Identity 1 (Proposition 1 of Rawat, 2026). For any function $Q(s,a)$, define $V(s)$ via the log-sum-exp and $\pi(a|s) = \exp\{(Q(s,a) - V(s))/\sigma\}$ via the softmax. Then $Q(s,a) - V(s) = \sigma \log \pi(a|s)$. The advantage function equals the scaled log choice probability under the softmax policy induced by Q.

Identity 2 (Proposition 2 of Rawat, 2026). If Q satisfies the Bellman fixed point $Q = \Lambda_\sigma(Q)$, where Λ_σ denotes the smoothed Bellman operator, then $\mathrm{td}(s,a) = r(s,a)$. The temporal difference equals the flow reward implied by Q through the inverse Bellman operator.

The equivalence. Substituting these identities into equation (5) yields the main result of Rawat (2026), stated here without proof.

$$\boxed{\min_Q \mathcal{L}(Q) \iff \max_Q \underbrace{\frac{1}{N}\sum_{i=1}^{N} \log \pi(a_i \mid s_i)}_{\text{conditional log-likelihood}} - \frac{1}{4\alpha\sigma}\sum_{i=1}^{N} \hat{r}_i^2} \tag{6}$$

The term $\mathrm{LL}(Q) = \sum_i \log \pi(a_i|s_i)$ is the conditional log-likelihood of observed actions under the softmax policy induced by Q, which is exactly the standard multinomial logit likelihood with Q-values playing the role of choice-specific utilities. The term $\hat{r}_i = \mathrm{td}(s_i, a_i)$ is the Bellman-implied reward at the i-th observation, so the penalty $\sum_i \hat{r}_i^2$ is an ℓ^2 norm on the magnitude of the implied reward at expert-visited state-action pairs. Minimizing the IQ-Learn objective is therefore equivalent to maximizing the fit to observed choices while penalizing the complexity of the reward function that rationalizes them.

Connection to NFXP. In NFXP (Rust, 1987; Iskhakov et al., 2016), the Bellman equation $Q = \Lambda_\sigma(Q)$ is imposed as a hard constraint. An inner loop solves the fixed point to machine precision before the outer loop evaluates the conditional log-likelihood and takes a gradient step in the structural parameters θ. In IQ-Learn, the Bellman equation enters as a soft constraint through the ℓ^2 penalty on temporal differences. The regularization parameter α governs the tradeoff between data fit and structural consistency. As $\alpha \to 0$ the penalty dominates and forces $\mathrm{td}(s,a) \to 0$ at observed state-action pairs, so IQ-Learn converges to the Bellman-constrained MLE that NFXP solves exactly. As $\alpha \to \infty$ the penalty vanishes and the estimator reduces to an unconstrained conditional logit that maximizes the likelihood of observed choices with no structural content from the dynamic model. The IQ-Learn regularization path therefore traces a continuous family of estimators between the unconstrained logit and the fully structural NFXP.

Connection to CCP estimation. Both CCP estimation (Hotz and Miller, 1993; Aguirregabiria and Mira, 2002) and IQ-Learn avoid the inner fixed-point loop that makes NFXP computationally expensive at high discount factors. CCP methods estimate choice probabilities nonparametrically in a first stage, use the Hotz-Miller inversion to recover continuation values from these probabilities, and then estimate the structural parameters θ in a second-stage conditional likelihood. IQ-Learn estimates the Q-function directly and recovers the implied reward via the inverse Bellman operator $r(s,a) = Q(s,a) - \beta \sum_{s'} V(s')\, p(s'|s,a)$. Both approaches replace the nested computation with a direct estimation step, but they differ in what is estimated in the first stage. CCP methods estimate the policy $\pi(a|s)$ and work backward to value functions, while IQ-Learn estimates $Q(s,a)$ and reads off the policy as a byproduct through the softmax.

Connection to GLADIUS. The ERM-IRL estimator of Kang et al. (2025) shares the same high-level structure as the tabular IQ-Learn rewrite in equation (6). Both objectives combine a conditional log-likelihood term with a Bellman-consistency penalty, and both avoid the inner fixed-point loop. The key difference lies in how the continuation value $\mathbb{E}[V(s') \mid s, a]$ is handled. Kang et al. (2025) work in a model-free setting where $p(s'|s, a)$ is unknown and introduce a dual variable $\zeta(s, a)$ that approximates the conditional expectation, turning the problem into a min-max optimization with provable convergence guarantees under their ERM framework. In the tabular DDC setting with known transitions, this expectation is computed exactly from the transition matrix, which eliminates the need for the dual variable and its associated double-sampling bias. The tabular IQ-Learn equivalence therefore reveals the simplest member of a family of penalized-likelihood estimators that also includes GLADIUS, with each member distinguished by its treatment of the transition kernel.